

Laboratorio: Módulos en JavaScript

Instrucciones de trabajo y ejecución

En este laboratorio trabajarás con módulos. Cada tarea se encuentra en un archivo independiente y debe exportar una función o clase con el nombre exacto que espera index.js.

Reglas importantes

- No modifiques index.js.
- Trabaja únicamente en los archivos task1.js, task2.js, task3.js, task4.js, task5.js, task6.js y task7.js.
- Los nombres de las funciones y clases deben coincidir exactamente con los indicados.
- Usa export para que index.js pueda importar el código de cada archivo.
- Guarda cada archivo antes de ejecutar las pruebas.

Qué debe exportar cada archivo

Archivo	Exportación requerida	Objetivo
task1.js	costCalculator	Calcular el costo de una transacción con cuota fija de \$3 y 1% de interés.
task2.js	FriendNames	Crear una clase que reciba tres nombres y guarde name1, name2 y name3.
task3.js	ageCalculator	Calcular la edad a partir de año, mes y día.
task4.js	FriendAge	Crear una clase con nombre y fecha de nacimiento, y un método returnAge().
task5.js	rubricPassFail	Devolver "Pass" o "Fail" según el puntaje.
task6.js	rubricExcellent	Devolver "Excellent" para puntajes mayores a 8; en otro caso, "Pass".
task7.js	rubricPerfect	Devolver "Perfect" para 11; en otro caso, "Pass".

Tareas

Tarea 1 - Costo de transacción

María quiere calcular cuánto debe pagar por una transacción. Cada operación tiene una cuota fija de \$3 y un interés de 1% (0.01). Crea y exporta una función llamada `costCalculator` que reciba un número y devuelva el total a pagar.

Tarea 2 - Nombres de amigos

Crea y exporta una clase llamada `FriendNames`. Su constructor debe recibir tres argumentos y crear exactamente las propiedades `name1`, `name2` y `name3`.

Tarea 3 - Cálculo de edad

Crea y exporta una función llamada `ageCalculator` que reciba año, mes y día y devuelva la edad calculada.

Tarea 4 - Nombre y edad

Crea y exporta una clase llamada `FriendAge`. Debe recibir `name`, `year`, `month` y `day`. Además, debe tener un método público llamado `returnAge()` que devuelva un texto con el formato: "<name> is <age> today!"

Tarea 5 - Aprobado o no aprobado

Crea y exporta `rubricPassFail`. Si el puntaje es mayor o igual a 5, devuelve "Pass"; si no, devuelve "Fail".

Tarea 6 - Excelente

Crea y exporta `rubricExcellent`. Si el puntaje es mayor que 8, devuelve "Excellent". Para un puntaje como 8, debe devolver "Pass".

Tarea 7 - Perfecto

Crea y exporta `rubricPerfect`. Si el puntaje es exactamente 11, devuelve "Perfect". Para un puntaje como 5, debe devolver "Pass".

Cómo ejecutar una tarea manualmente

`index.js` recibe primero el número de la tarea y después los datos de entrada. Puedes probar cada tarea desde la terminal.

```
node index.js 1 124
```

```
node index.js 2 Karim Uli Georgina
```

```
node index.js 3 2001 12 25
```

```
node index.js 4 Kimi 1998 11 5
```

```
node index.js 5 6
```

```
node index.js 6 9
```

```
node index.js 7 11
```

Cómo ejecutar todas las pruebas

Si pytest ya está instalado, ejecuta:

```
python3 -m pytest -v
```

Las pruebas ejecutan automáticamente comandos como `node index.js 1 124` y comparan la salida obtenida con la salida esperada.

Resultados esperados de las pruebas

- Tarea 1 con 124 → 128.24
- Tarea 2 con Karim, Uli y Georgina → objeto FriendNames con esas tres propiedades
- Tarea 3 → Successful
- Tarea 4 → Successful
- Tarea 5 con 6 → Pass
- Tarea 5 con 4 → Fail
- Tarea 6 con 9 → Excellent
- Tarea 6 con 8 → Pass
- Tarea 7 con 11 → Perfect
- Tarea 7 con 5 → Pass

Si una prueba falla

- Lee primero el error que aparece en la terminal.
- Si aparece "ReferenceError: ... is not defined", revisa que el parámetro exista dentro de los paréntesis de la función.
- Comprueba que el nombre exportado sea exactamente el que index.js espera.
- Verifica que el archivo esté guardado.
- Prueba primero la tarea con `node index.js ...` antes de volver a ejecutar pytest.